

$[i*j \text{ for } i \text{ in range}(1,4)] \text{ for } j \text{ in range}(1,4)]$

$L1 = [1, 2, 3, 4]$

$L2 = [5, 6, 7, 8]$

$[i*j \text{ for } i \text{ in } L1 \text{ for } j \text{ in } L2]$

o/p →

Zip

zip() ← function

$L1 \leftarrow$
 $L2 \leftarrow$

list(zip(L1, L2))

Disadvantage
→ slow

Problem-01

$L = [1, \dots, N]$

L-even

L-odd

$L = [10, 15, 11, 40]$

$L\text{-even} = [10, 40]$

$L\text{-odd} = [11, 15]$

Problem-02

$L1 \leftarrow [1, 2, 3]$

$L2 \leftarrow [4, 5, 6, 7]$

merge

$L = [1, 2, 3, 4, 5, 6]$

$L1 \oplus L2$

Problem-03

$L \rightarrow [1, 2, 3, 4, 5]$

user input

3

replace

user input

500

Problem-04

$L \rightarrow [[1, 2, 3], [4, 5, 6], [7, 8, 9]]$

2D → 1D → $[1, 2, 3, 4, 5, 6, 7, 8, 9]$

$L[0][1]$

	col	0	1	2
row 0	1	2	3	
1	4	5	6	
2	7	8	9	

$L = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]$

$L_{-2D} = []$

I will traverse each row:
for element in row:
 $L_{-1D}.append(element)$

0 →	①	②	③
1 →	4	5	6
2 →	7	8	9

Problem - 05

$L = [1, 2, 1, 2, 3, 4, 5, 3, 4]$

→ "duplicated value remove"

Hint: "membership operator"
 → in / not in

$L1 = []$

→ for each value in L
 → value not in L1:
 append

Tuple

$t1 = \begin{matrix} -4 & -3 & -2 & -1 \\ (1, 2, 3, 4) \\ 0 & 1 & 2 & 3 \end{matrix}$

$list('hello') \rightarrow$

h	e	l	l	o
0	1	2	3	4

$t1 = ()$

$t1 = ('hello',)$

$t2 = (1, 2.5, True, [1, 2, 3])$

$t3 = \begin{matrix} (1, 2.5, (1, 2, 3)) \\ -3 \quad -2 \quad -1 \end{matrix}$

$tuple('hello') \rightarrow ('h', 'e', 'l', 'l', 'o')$

Access
 $t1[0]$

$t3[-1][0] \Rightarrow 1$

Edit

→ not possible

Add

→ not possible

Delete

→ not possible

$t = (1, 2, 3, 4, 5)$
-4 -3 -2 -1

$t[-1:-4:-1] \rightarrow 5, 4, 3$

Operations

$t_1 = (1, 2, 3) \leftarrow$
 $t_2 = (4, 5, 6) \leftarrow$

$(t_1 + t_2) \leftarrow (1, 2, 3, 4, 5, 6)$

↑
new tuple

$(t_1 * 3) \leftarrow (1, 2, 3, 1, 2, 3, 1, 2, 3) \leftarrow$

functions

$t = (1, 2, 3, 4)$

$\text{len}(t) \leftarrow 4$

$\text{sum}(t) \leftarrow$

$\text{min}(t)$

$\text{max}(t)$

$\text{sorted}(t) \leftarrow \text{new tuple}$

$\text{sorted}(t, \text{reverse} = \text{True}) \leftarrow \text{new tuple}$

for i in t:
print(i)
1
2
3

for i in range(len(t)):
print(i)
print(t[i])

$t.\text{index}(\text{value}) \leftarrow$

List vs Tuple

Syntax

Mutability

Speed

→ less memory

Memory →

Special syntax

$a, b, c = (1, 2, 3)$
print(a, b, c)

$a, b = (1, 2, 3) // \text{error}$
X

$a = 10$
 $b = 20$
 $a, b = b, a$

$a, b, *others = (1, 2, 3, 4)$
↓
[3, 4]

Zip

$a = (1, 2, 3, 4)$

$b = (5, 6, 7, 8)$

$\text{tuple}(\text{zip}(a, b))$

$((1, 5), (2, 6), (3, 7), (4, 8))$

Sets

$S = \{1, 2, 3\}$

$S = \{1, 2, 3\}$

create

$S = \text{set}() \rightarrow \{\}$

$S_1 = \{1, 2, 3\}$

$S_2 = \{3, 2, 1\}$

$S_1 == S_2$

Access

$S[0]$

$S[0:2]$

not possible

Edit

$S[0] = 10 \rightarrow$ not possible

Add

$S = \{1, 2, 3, 4\}$

add

$S.add(5) \rightarrow$

update

$S.update([5, 6, 7]) \rightarrow \{1, 2, 3, 4, 5, 6, 7\}$

Recursion

① set

② functions ✓

③ OOP (object oriented programming)

↳ class & object

↳ Encapsulation, Abstraction, Inheritance

④ file handling

⑤

Exception handling

*